

Follow the Gradient

VLC = $\iiint$ Vision
Learning & Control

Differentiable Matrix Completion

Jonathon Hare

Vision, Learning and Control
University of Southampton

Recap: Singular Value Decomposition and its applications

Let's now change direction — we're going to look at an early success story resulting from using some differentiation and the Singular Value Decomposition (SVD).

For complex $\mathbf{A}$:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$$

where $\mathbf{V}^*$ is the *conjugate transpose* of $\mathbf{V}$.

For real $\mathbf{A}$:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of $\mathbf{\Sigma}$ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.
 - Dimensionality reduction
 - ...use to compute PCA
 - Computing the Moore-Penrose Pseudoinverse
 - for real $\mathbf{A}$: $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ where $\mathbf{\Sigma}^+$ is formed by taking the reciprocal of every non-zero diagonal element and transposing the result.
 - Low-rank approximation and matrix completion
 - if you take the ρ columns of $\mathbf{U}$, and the ρ rows of $\mathbf{V}^\top$ corresponding to the ρ largest singular values, you can form the matrix $\mathbf{A}_\rho = \mathbf{U}_\rho\mathbf{\Sigma}_\rho\mathbf{V}_\rho^\top$ which will be the *best* rank- ρ approximation of the original $\mathbf{A}$ in terms of the Frobenius norm.

Example: Computing SVD using gradients - The Netflix Challenge

- There are many standard ways of computing the SVD:
 - e.g. 'Power iteration', or 'Arnoldi iteration' or 'Lanczos algorithm' coupled with the 'Gram-Schmidt process' for orthonormalisation
- but, these don't necessarily scale up to really big problems
 - e.g. computing the SVD of a sparse matrix with 17770 rows, 480189 columns and 100480507 non-zero entries!
 - this corresponds to the data provided by Netflix when they launched the *Netflix Challenge* in 2006.
- OK, so what can you do?
 - The 'Simon Funk' solution: realise that there is a really simple (and quick) way to compute the SVD by following gradients...

Example: Computing SVD using gradients - The Netflix Challenge

Deriving a gradient-descent solution to SVD

- One of the definitions of rank- ρ SVD of a matrix $\mathbf{A}$ is that it minimises reconstruction error in terms of the Frobenius norm.
- Without loss of generality we can write SVD as a 2-matrix decomposition $\mathbf{A} = \hat{\mathbf{U}}\hat{\mathbf{V}}^T$ by rolling in the square roots of Σ to both $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$: $\hat{\mathbf{U}} = \mathbf{U}\Sigma^{0.5}$ and $\hat{\mathbf{V}}^T = \Sigma^{0.5}\mathbf{V}^T$.
- Then we can define the decomposition as finding $\min_{\hat{\mathbf{U}}, \hat{\mathbf{V}}} (\|\mathbf{A} - \hat{\mathbf{U}}\hat{\mathbf{V}}^T\|_F^2)$

Example: Computing SVD using gradients - The Netflix Challenge

Deriving a gradient-descent solution to SVD

Start by expanding our optimisation problem:

$$\begin{aligned}\min_{\hat{\mathbf{U}}, \hat{\mathbf{V}}}(\|\mathbf{A} - \hat{\mathbf{U}}\hat{\mathbf{V}}^\top\|_F^2) &= \min_{\hat{\mathbf{U}}, \hat{\mathbf{V}}}(\sum_r \sum_c (A_{rc} - \hat{U}_r \hat{V}_{:,c}^\top)^2) \\ &= \min_{\hat{\mathbf{U}}, \hat{\mathbf{V}}}(\sum_r \sum_c (A_{rc} - \sum_{p=1}^{\rho} \hat{U}_{rp} \hat{V}_{cp})^2)\end{aligned}$$

Let $e_{rc} = A_{rc} - \sum_{p=1}^{\rho} \hat{U}_{rp} \hat{V}_{cp}$ denote the error. Then, our problem becomes:

$$\text{Minimise } \mathcal{L} = \sum_r \sum_c e_{rc}^2$$

We can then differentiate with respect to specific variables $\hat{U}_{rq}$ and $\hat{V}_{cq}$

Example: Computing SVD using gradients - The Netflix Challenge

Deriving a gradient-descent solution to SVD

We can then differentiate with respect to specific variables $\hat{U}_{rq}$ and $\hat{V}_{cq}$:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \hat{U}_{rq}} &= \sum_r \sum_c 2e_{rc} \frac{\partial e}{\partial \hat{U}_{rq}} = -2 \sum_r \sum_c \hat{V}_{cq} e_{rc} \\ \frac{\partial \mathcal{L}}{\partial \hat{V}_{cq}} &= \sum_r \sum_c 2e_{rc} \frac{\partial e}{\partial \hat{V}_{cq}} = -2 \sum_r \sum_c \hat{U}_{rq} e_{rc}\end{aligned}$$

and use this as the basis for a gradient descent algorithm:

$$\begin{aligned}\hat{U}_{rq} &\Leftarrow \hat{U}_{rq} + \lambda \sum_r \sum_c \hat{V}_{cq} e_{rc} \\ \hat{V}_{cq} &\Leftarrow \hat{V}_{cq} + \lambda \sum_r \sum_c \hat{U}_{rq} e_{rc}\end{aligned}$$

Example: Computing SVD using gradients - The Netflix Challenge

Deriving a gradient-descent solution to SVD

- A stochastic version of this algorithm (updates on one single item of $\mathbf{A}$ at a time) helped win the Netflix Challenge competition in 2009.
- It was both *fast* and *memory efficient*